


# Cursul 3
## Dezvoltarea programelor

---

Utilizarea Sistemelor de Operare (USO)
9 noiembrie 2017

Departamentul de Calculatoare

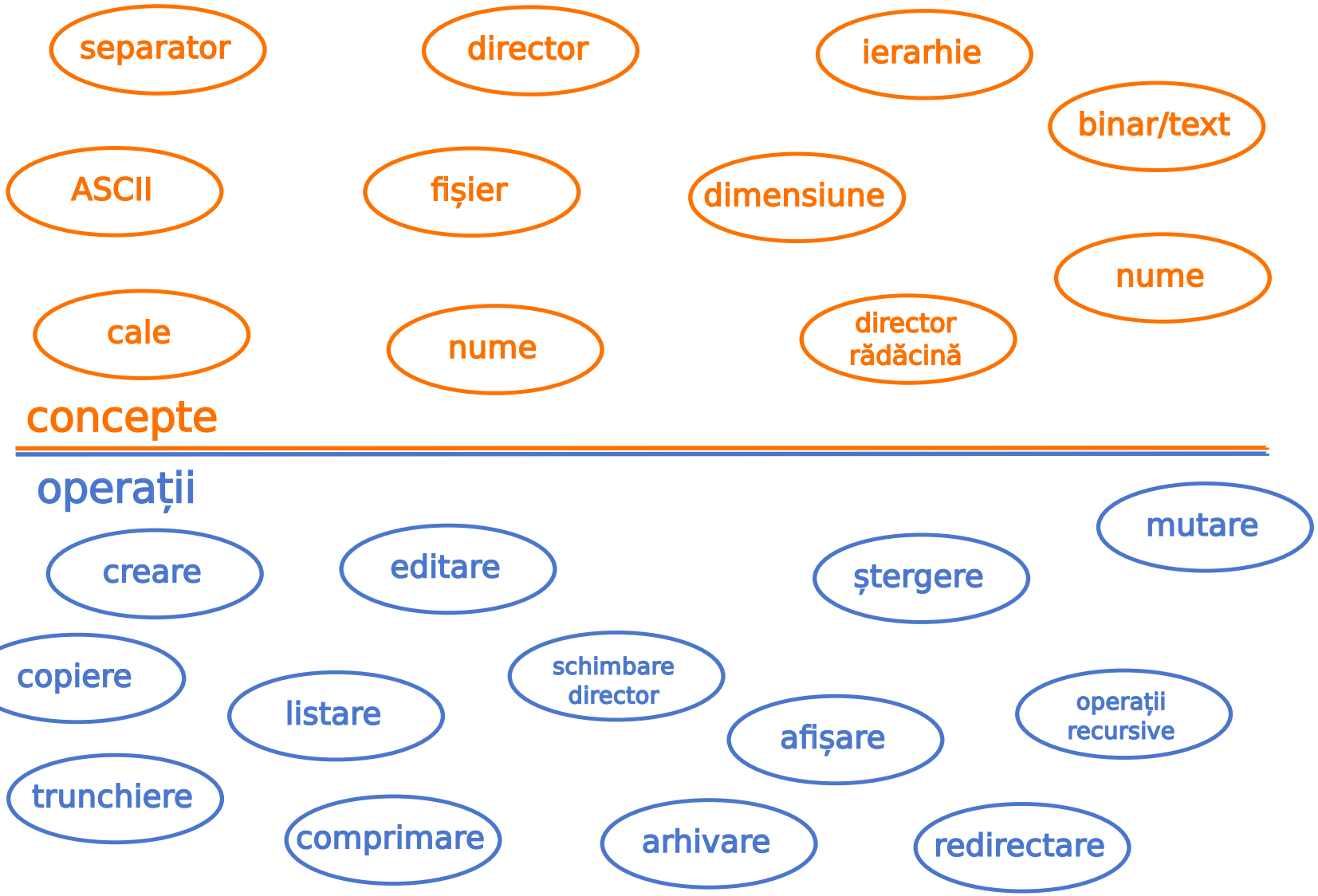
# Moto

*There are only two kinds of programming languages: those people always bitch about and those nobody uses.*

*Bjarne Stroustrup*

*One of the main causes of the fall of the Roman Empire was that – lacking zero – they had no way to indicate successful termination of their C programs.*

*Robert Firth*


separator
director
ierarhie
binar/text
ASCII
fișier
dimensiune
nume
cale
nume
director rădăcină
concepte
operații
mutare
creare
editare
ștergere
copiere
schimbare director
listare
operații recursive
afișare
trunchiere
comprimare
arhivare
redirectare

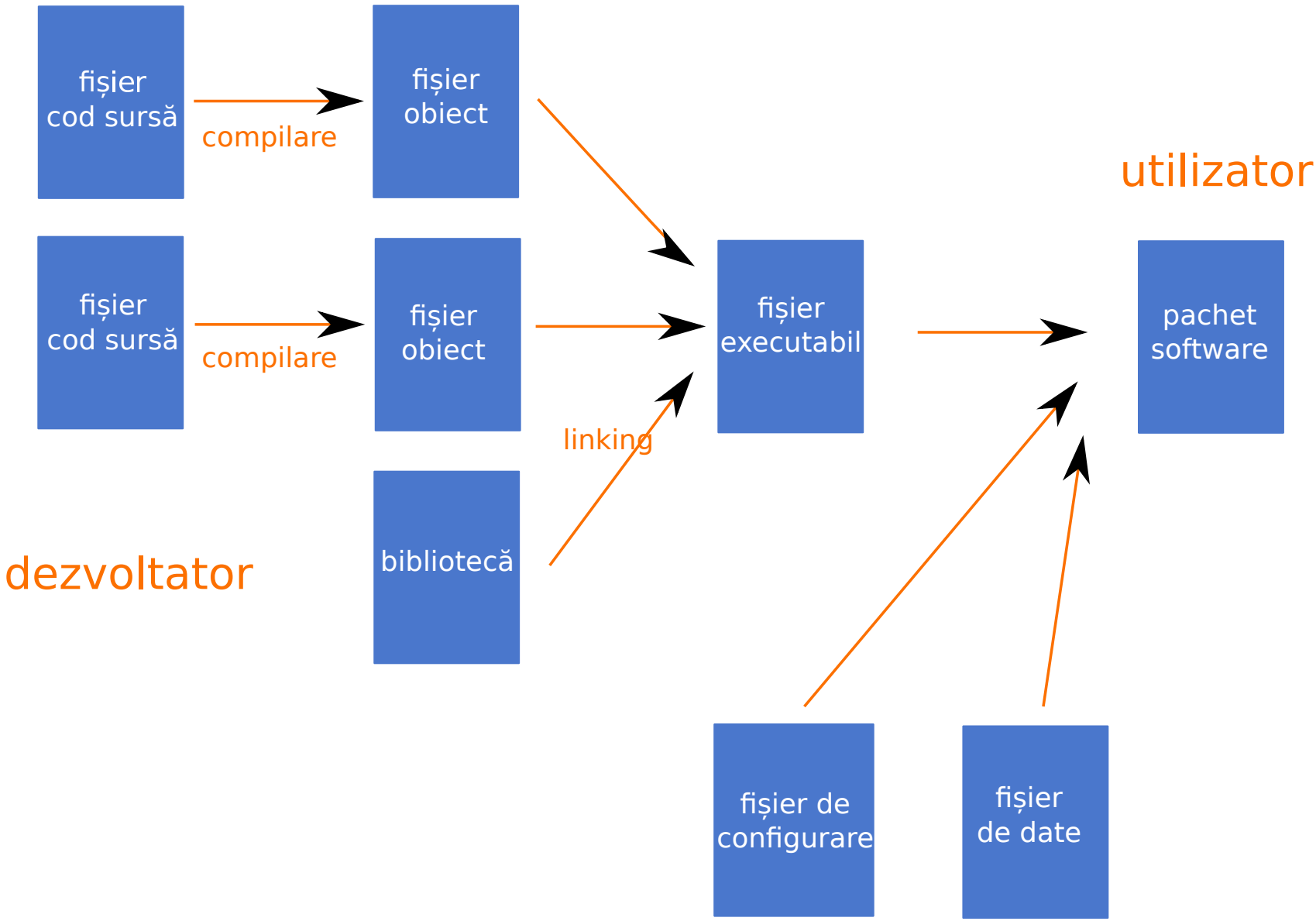



http://www.geekarmy.com/geeks/2906/file-transferring/

# Remember: WoUSO v11

- începe marți, 10 octombrie 2017
- folosiți contul de cs.curs.pub.ro pentru autentificare
- urmăriți forumuri și pagina de Facebook

- Suport (Introducere în sisteme de operare)
  - Capitolul 11 – Compilare și linking
  - Capitolul 14 – Utilitare pentru dezvoltare


fișier
cod sursă
compilare
fișier
obiect
fișier
cod sursă
compilare
fișier
obiect
bibliotecă
linking
fișier
executabil
pachet
software
utilizator
dezvoltator
fișier de
configurare
fișier
de date

# Operațiile utilizatorului cu aplicațiile

- instalare
- dezinstalare
- parcurgere documentație
- configurare și personalizare
- rulare (cu diverse opțiuni)
- dezvoltatorul trebuie să-i facă viața cât mai ușoară utilizatorului

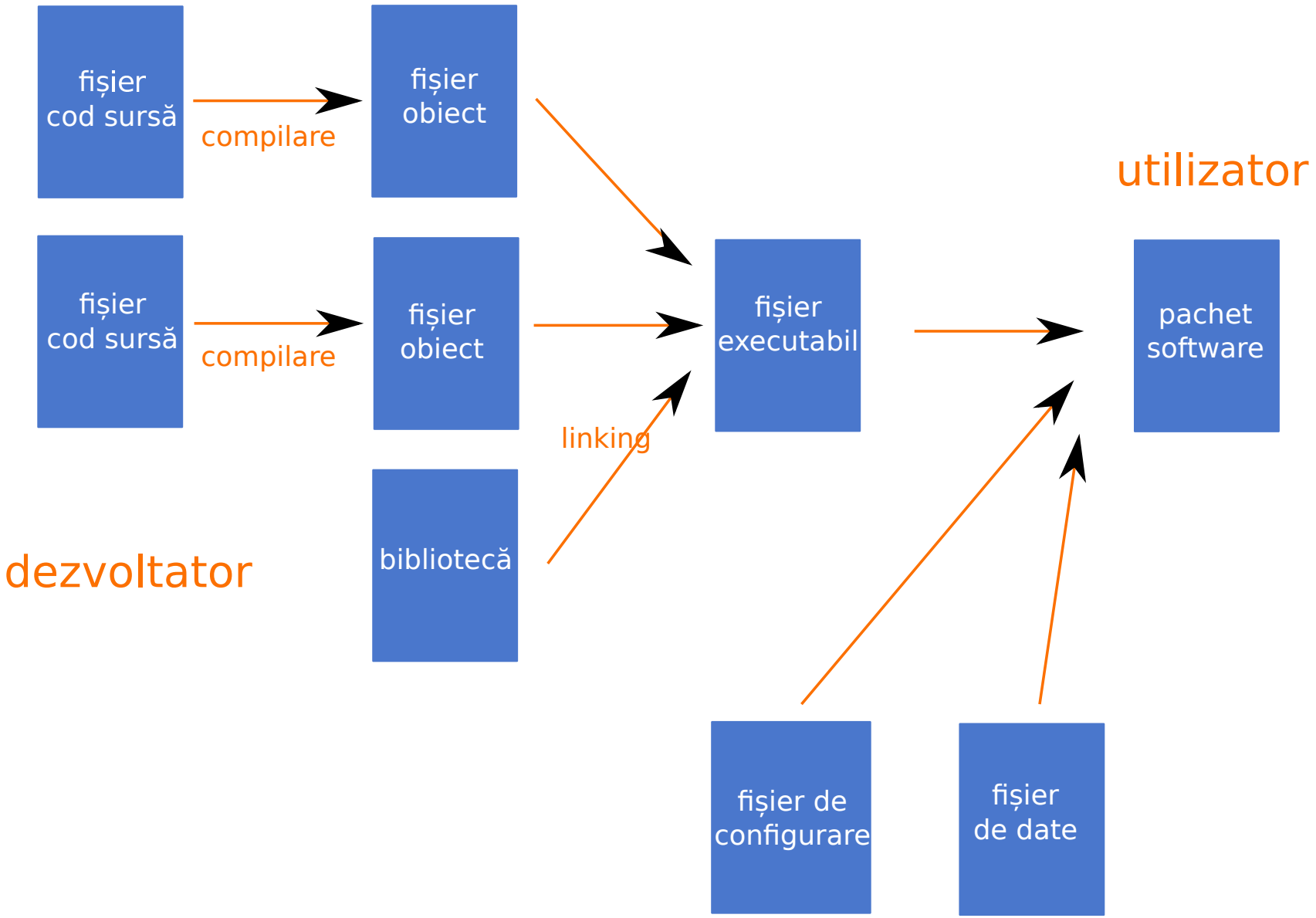
- arhive cu fișierele necesare pentru instarea aplicației
- fișiere de date, fișiere de configurare, fișiere executabile
- fișierele executabile sunt folosite pentru a rula aplicația
- utilizatorul instalează pachetele software și apoi folosește fișierele din acestea

**Conținutul pachetului `tcpdump`**

```
user@host:~$ dpkg -L tcpdump
[...]
/usr/share/man/man8/tcpdump.8.gz    # pagina de manual
[...]
/usr/sbin/tcpdump                   # executabilul din pachet
```

- folosite pentru rularea aplicației
- dublu click în GUI, rulare de comandă în linia de comandă
- obținute din codul sursă
- în Unix se găsesc, în general în `/bin`, `/sbin`, `/usr/bin` și `/usr/sbin`
- pot fi binare sau text (ASCII)
  - binare pentru cod sursă compilat (C, C++, Java)
    - `/bin/ls`, `/usr/bin/vim`
  - text pentru cod sursă interpretate (Perl, Python, Ruby)
    - `ls /bin/* | grep ASCII`
    - `ls /usr/bin/* | grep ASCII`

# De la cod sursă la pachet software



# Cod sursă pentru un limbaj

- un fișier al unui program este denumit fișier cod sursă (*source code*)
- codul sursă este codul citibil al unui limbaj de programare: format text
- cod sursă: instrucțiuni într-un limbaj
- codul sursă respectă sintaxa acelui limbaj
- pentru a înțelege un program este necesară cunoașterea sintaxei limbajului
- codul sursă este scris de programator

- folosite pentru scrierea codului sursă
- editoare
  - scriu în general text, printre care și cod sursă
  - Vim, Emacs, Notepad++, Sublime Text
- IDE: *Integrated Development Environment*
  - integrează editor, compilator, debugger, biblioteci
  - Eclipse, Visual Studio, Code::Blocks, NetBeans
- facilități
  - syntax highlighting, auto indentation, utilitare pentru debugging integrate
  - code folding, code completion (autocompletion)

# Coding Style

- convenții și recomandări pentru scrierea codului sursă
- ușurință în a înțelege codul sursă
- consecvență
- în general cu suport din partea editoarelor și IDE-urilor
- aspecte acoperite
  - indentare
  - nume de funcții, variabile, constante, macro-uri
  - spațiere
  - comentarii de cod
- Steve Oualline: C Elements of Style
- Python PEP 8

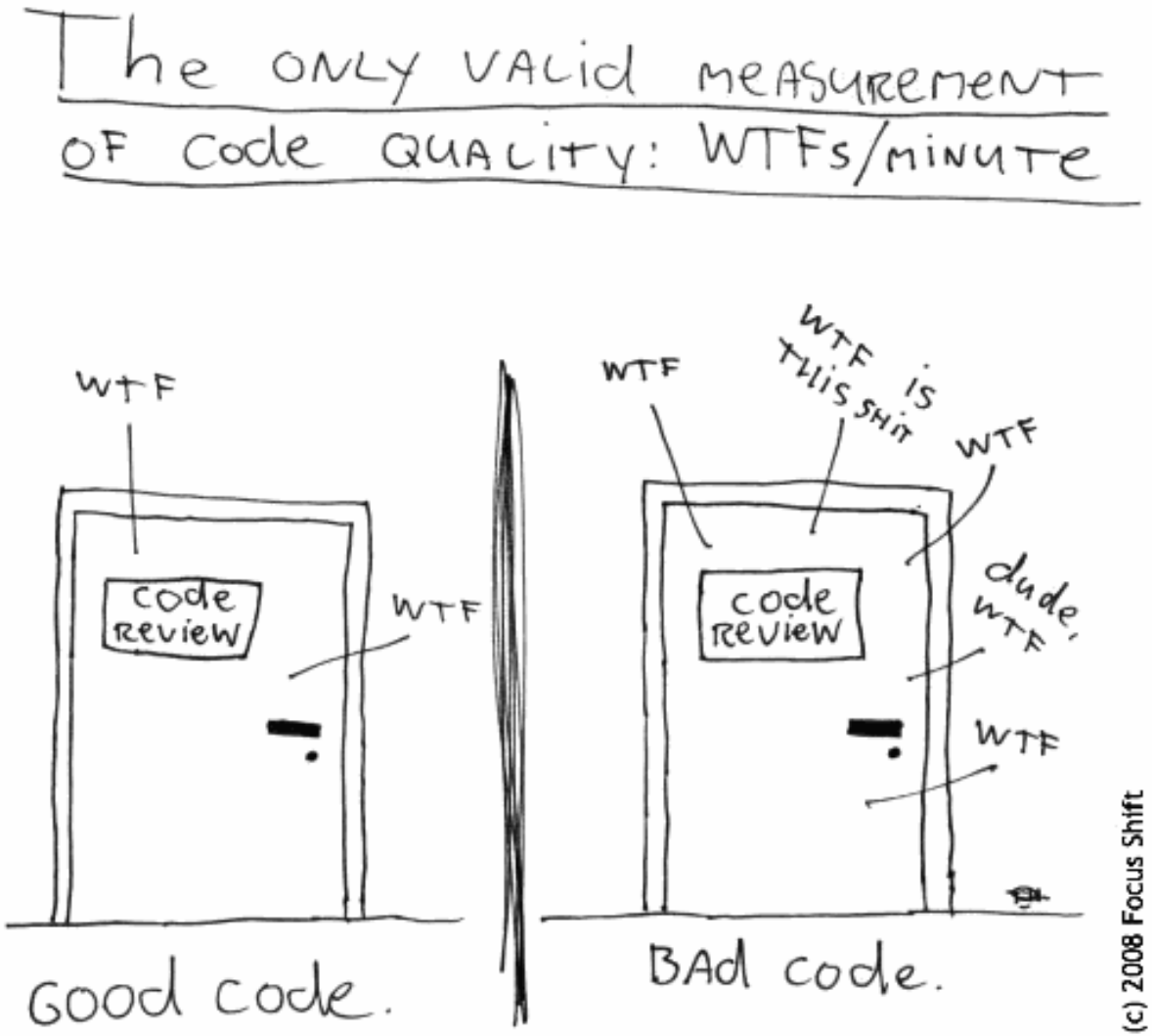
# Bad Code

```
main(){
  int extra; char *len="prime";
int num;
     int is_prime,i;
  for(i=0;i<10;i++){printf("Introduceti numar:");
                scanf("%d", &num);
      is_prime=prime(num);
        if(is_prime)
            printf("is %s\n",len); else printf("is not %s\n",len);
     extra=get_sum(num);
   printf("sum (1..%d) is %d\n",num,extra);
 }
}
```

```c
int main(void)
{
        size_t i, num;

        for (i = 0; i < NUM_ROUNDS; i++) {
                printf("Introduceti numar: ");
                scanf("%zu", &num);
                if (check_if_prime(num))
                        printf("%zu e numar prim.\n", num);
                else
                        printf("%zu nu e numar prim.\n", num);
                printf("Suma (1..%zu) este %zu\n", num, compute_sum_until(num));
        }

        return 0;
}
```




http://blog.codinghorror.com/whos-your-coding-buddy/

- un fișier cod sursă poate fi compilat sau interpretat
- deosebirea între compilare și interpretare
  - compilare: codul sursă este translatat de un program denumit compilator în cod mașină (*machine code*), după care poate fi executat
  - interpretare: un program este executat direct din cod sursă prin intermediul unui interpretor
- codul mașină este cel care rulează, în final, pe procesorul sistemului
- compilatoare și interpretoare
  - GCC (GNU Compiler Collection) – compilator de C, C++, Ada, Fortran
  - MSVC (Microsoft Visual C) – compilator de C, C++
  - GCL (GNU Common Lisp) – interpretor de Common Lisp
  - Python: interpretor de Python
  - Perl: interpretor de Perl

# Linking

- în urma procesului de compilare, din modulul cod sursă rezultă un modul obiect
- modulul obiect este fișier binar (`.o`, `.obj`): conține cod mașină
- mai multe fișiere obiect (+biblioteci) sunt legate (*linked*) într-un fișier executabil: conține cod mașina
- cod sursă compilat $\rightarrow$ modul obiect (cod mașină)
- module obiect și biblioteci linkate $\rightarrow$ executabil

# De ce programare în limbajul C?

- limbaj foarte cunoscut
- limbaj foarte puternic
- limbaj aproape de hardware și sistemul de operare, eficient, rapid
- documentație, tutoriale, cărți

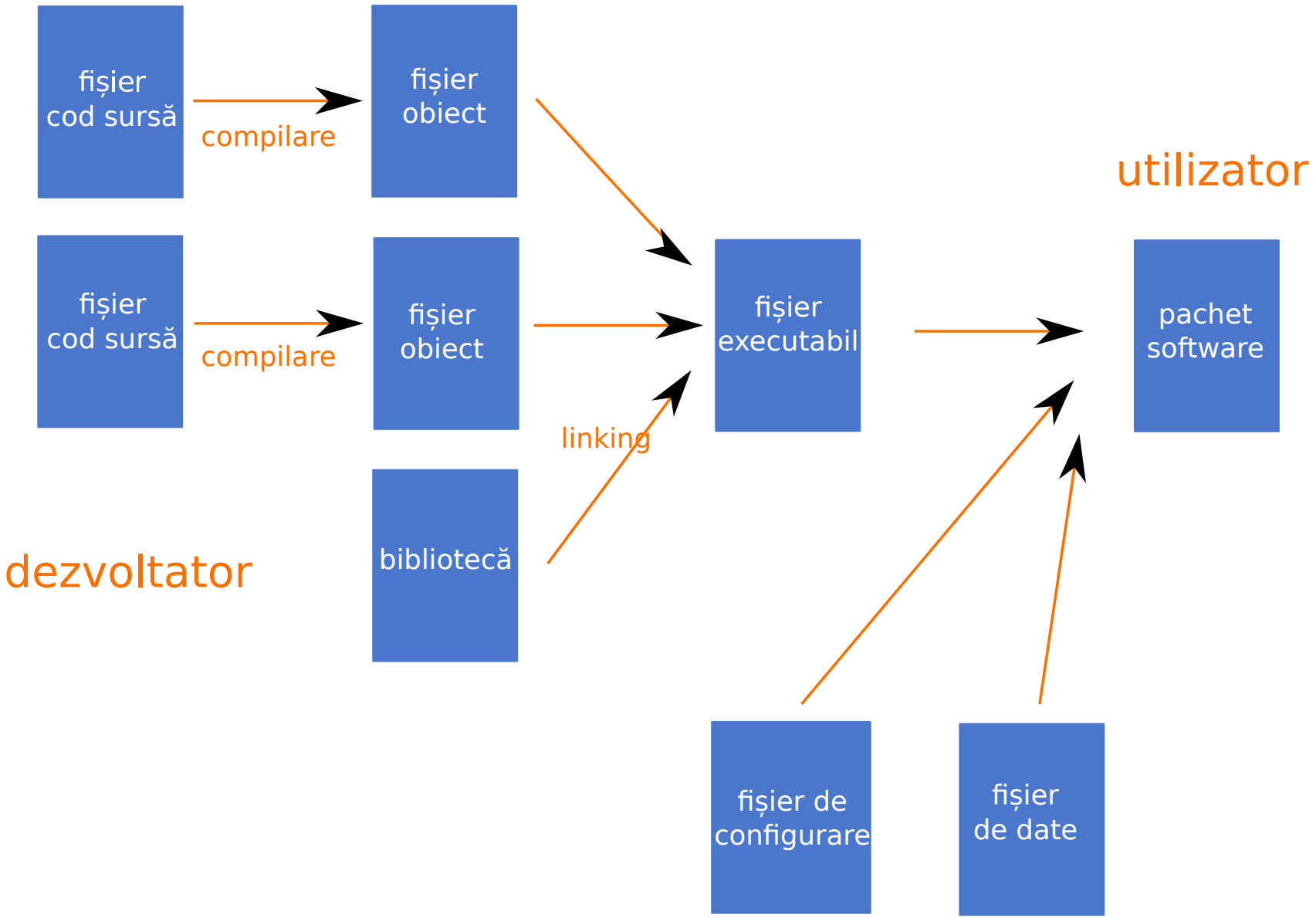
# De ce nu programare în limbajul C?

- limbaj relativ greu de învățat
- limbaj periculos, necesită experiență
- limbaj mai puțin expresiv
- durează mult timp să dezvolți o aplicație
- posibile probleme de portabilitate (Linux, Windows, Mac OS X, Android etc.)

# Procesul de dezvoltare pentru programe C

- scriere de fișiere cod sursă: surse (`.c`) și headere (`.h`)
- urmărirea unui coding style
- compilarea fișierelor cod sursă în fișiere obiect
- linkarea fișierelor cod obiect și a bibliotecilor în fișier executabil
- pe Unix/Linux, pentru compilare și linking folosim `gcc` (`GNU C Compiler`)

# Reminder: De la cod sursă la pachet software



## Compilare cu gcc

```
user@host$ ls -F
hello-world.c
user@host$ gcc hello-world.c
user@host$ ls -F
a.out*  hello-world.c
user@host$ ./a.out
Hello, World!
user@host$ rm a.out
user@host$ ls -F
hello-world.c
user@host$ gcc -Wall hello-world.c -o hello-world
user@host$ ls -F
hello-world*  hello-world.c
user@host$ ./hello-world
Hello, World!
```

## Compilare cu gcc

```
user@host$ ls -F
hello-world.c
user@host$ gcc -Wall -c hello-world.c
user@host$ ls -F
hello-world.c  hello-world.o
user@host$ gcc hello-world.o -o hello-world
user@host$ ls -F
hello-world*  hello-world.c  hello-world.o
user@host$ ./hello-world
Hello, World!
```

- Do one thing, do one thing well!
- funcționalitățile diferite intră în fișiere sursă diferite
- evităm îngrămădirea funcționalităților într-un singur fișier
- fiecare fișier este numit ,,modul" (*module*)sau ,,unitate de compilare" (*compilation unit*)

## Compilare cu gcc din mai multe surse

```
user@host$ ls -F
debug.h  http_reply_once.c  sock_util.c  sock_util.h  util.h
user@host$ gcc -Wall -c sock_util.c
user@host$ gcc -Wall -c http_reply_once.c
user@host$ ls -F
debug.h            http_reply_once.o  sock_util.h  util.h
http_reply_once.c  sock_util.c        sock_util.o
user@host$ gcc http_reply_once.o sock_util.o -o http_reply_once
user@host$ ls -F
debug.h           http_reply_once.c  sock_util.c  sock_util.o
http_reply_once*  http_reply_once.o  sock_util.h  util.h
```

# Procesul de build

- *building*: obținerea unui executabil, a unui set de executabile sau a unui pachet software din fișiere cod sursă
- o versiune de pachet construită se mai cheamă și un *build*
- pentru programe scrise în C înseamnă compilare, linking, împachetare

- cele care permit automatizarea procesului de build
- make: folosit foarte mult în lumea Unix/Linux
- Ant, Maven: folosite pentru Java
- Scons: scris în Python
- Rake: folosit pentru Ruby
- altele

# Make

- sistem de build și automatizare
- folosit îndeosebi pentru automatizare procesului de compilare (dar nu numai)
- se scriu reguli într-un fișier denumit în general `Makefile` (un fel de fișier cu ,,rețete")
- se rulează comenzile aferente regulilor, în mod automat, folosind comanda `make`
- se obțin *target*-uri (în general fișiere executabile) care depind de *dependențe* (în general fișiere cod sursă)

```
1 hello-world: hello-world.c
2         gcc -Wall hello-world.c -o hello-world
```

## Compilare folosind make pentru fișierul Makefile de mai sus

```
user@host$ ls -F
hello-world.c
user@host$ make
gcc -Wall hello-world.c -o hello-world
user@host$ ls -F
hello-world*  hello-world.c
```

```
1 all: hello-world
2
3 hello-world: hello-world.c
4         gcc -Wall hello-world.c -o hello-world
5
6 clean:
7         rm -f hello-world hello-world.o
```

## Compilare folosind `make` pentru fișierul Makefile de mai sus

```
user@host$ ls -F
hello-world.c
user@host$ make
gcc -Wall hello-world.c -o hello-world
user@host$ ls -F
hello-world*  hello-world.c
```

# Folosire elegantă a Make

```
1  .PHONY: all clean
2
3  all: hello-world
4
5  hello-world: hello-world.o
6          gcc hello-world.o -o hello-world
7
8  hello-world.o: hello-world.c
9          gcc -Wall -c hello-world.c
10
11 clean:
12         -rm -f hello-world hello-world.o
13         -rm -f *~
```

## Compilare folosind `make` pentru fișierul Makefile de mai sus

```
user@host$ ls -F
hello-world.c
user@host$ make -f Makefile.elegant
gcc -Wall -c hello-world.c
gcc hello-world.o -o hello-world
user@host$ ls -F
hello-world*  hello-world.c  hello-world.o
```

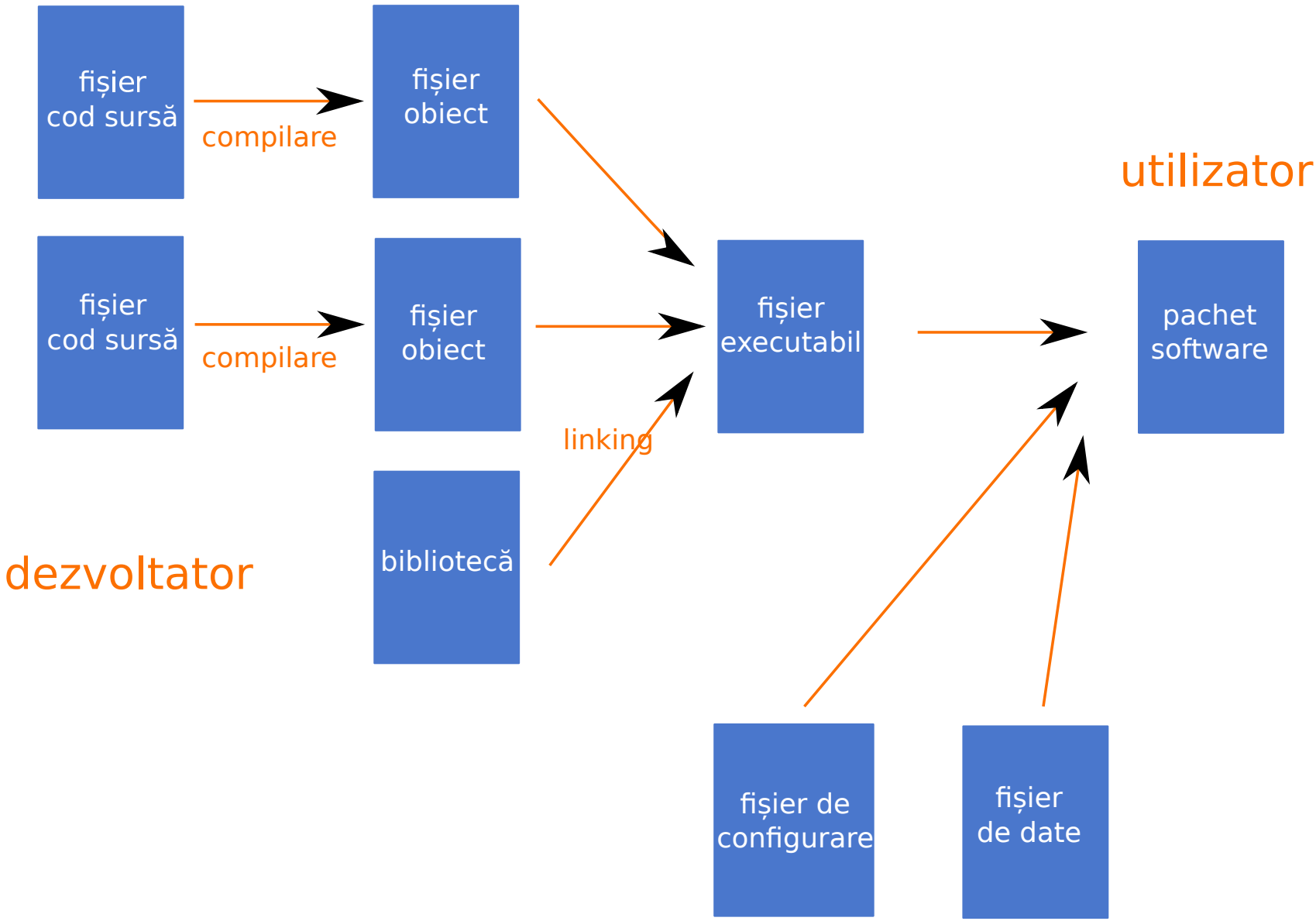
# Folosire elegantă a Make pentru surse multiple

```
1  .PHONY: all clean
2
3  all: http_reply_once
4
5  http_reply_once: http_reply_once.o sock_util.o
6          gcc http_reply_once.o sock_util.o -o http_reply_once
7
8  http_reply_once.o: http_reply_once.c util.h debug.h sock_util.h
9          gcc -Wall -c http_reply_once.c
10
11 sock_util.o: sock_util.c util.h debug.h sock_util.h
12         gcc -Wall -c sock_util.c
13
14 clean:
15         -rm -f http_reply_once http_reply_once.o sock_util.o
16         -rm -f *~
```

## Compilare folosind `make` pentru fișierul Makefile de mai sus

```
user@host$ ls -F
Makefile  debug.h  http_reply_once.c  sock_util.c  sock_util.h  util.h
user@host$ make
gcc -Wall -c http_reply_once.c
gcc -Wall -c sock_util.c
gcc http_reply_once.o sock_util.o -o http_reply_once
user@host$ ls -F
Makefile  http_reply_once*   http_reply_once.o  sock_util.h  util.h
debug.h   http_reply_once.c  sock_util.c        sock_util.o
```

# De la dezvoltator la utilizator



GCC *GNU C Compiler*

- compilatorul implicit pe distribuțiile Linux
- compilează și linkează cod sursă în C

Make

- utilitar pentru automatizarea procesului de *build*
- reguli, target-uri, dependențe

# Resurse utile

- http://www.oualline.com/style/index.html
- http://www.gnu.org/software/make/
- http://www.gnu.org/software/libc/manual/
- http://git-scm.com/
- http://gitimmersion.com/
- http://www.moolenaar.net/habits.html

- cod sursă
- cod mașină
- *coding style*
- editor
- IDE
- pachet software
- fișier executabil
- fișier obiect
- compilare
- linking
- limbajul C
- `gcc`
- modularizare
- sistem de build
- `make`
- `Makefile`

# Compilers: Principles, Techniques and Tools

- The Dragon Book
- Aho, Sethi, Ullman
- 2nd Edition, 2006
- cartea de bază pentru toate cursurile de compilatoare din universități
- expunere exhaustivă a analizei sintactice, semnatice și parserelor

# Guido van Rossum

- inventatorul limbajului de programare Python
- Benevolent Dictator for Life (BDFL) pentru Python
- a activat la Google în perioada 2005-2012
- din 2013 lucrează la Dropbox

# Google

- google.com – cel mai folosit site din lume
- fondată de Larry Page și Sergey Brin
- lansată în 1998
- inițial: search engine + advertising
- aplicații web
- Android
- Chrome
- YouTube

# Valgrind

- `http://valgrind.org`
- detectarea de probleme la rulare (*runtime*)
- în principal folosit pentru probleme de lucru cu memoria
- Linux și Darwin (Mac OS X)
- un engine peste care rulează componente dedicate: memcheck (implicit), cachegrind, callgrind, helgrind